Detecting Initial /g/, /b/, /d/, and /z/ in MSWC English:
A Course-Constrained Spectral Template Method with Extended Baselines

Heng Qiao

April 29, 2026

Abstract

This report reformulates VE216 Project Topic 2 as a controlled initial-consonant detection problem. From MSWC English, the subset keeps only words whose first letter is `g`, `b`, `d`, or `z` and whose first non-vowel ARPABET phone is the matching consonant. The resulting corpus contains 60 words and 3000 utterances. The main method, `strict_course_fft`, uses short-time framing, FFT magnitude spectra, maximum-energy blocks, normalized correlation, and a sliding maximum score. It reaches 0.343 exact-match accuracy on the four-way test set. An extended template method, `course_filterbank_corr`, adds log-mel localization, positive-negative templates, and envelope autocorrelation, raising exact-match accuracy to 0.550. A feature MLP reaches 0.527, while a small CNN reaches 0.320. The results show that /z/ is the easiest class because its frication persists over time, while /g/ is the hardest because its burst is short and sensitive to alignment. The main value of the study is an interpretable signal-processing pipeline that makes the acoustic evidence visible before comparing stronger baselines.

Keywords: speech analysis; voiced consonants; short-time Fourier transform; template matching; keyword spotting; MSWC.

1 Introduction and Problem Formulation

The original project asks for spectral analysis of voiced consonants and a program that detects selected letters in spoken words. This report treats the task as four-way initial-consonant recognition rather than general word recognition. Each recording is a one-second, 16 kHz waveform $x_i[n]$. The class set is

$$C = \{g, b, d, z\}, \qquad y_i \in \{0,1\}^{|C|}.$$

Every retained sample has exactly one target initial consonant. The code still uses a four-output interface so that the template methods, MLP, and CNN can share the same evaluation path. A method produces a score vector $s_i \in \mathbb{R}^{|C|}$, and the final label is decoded as

$$\hat{c}_i = \arg\max_{c \in C} s_{i,c}, \qquad \hat{y}_{i,c} = \mathbf{1}[c = \hat{c}_i].$$

Per-class thresholds are estimated on the development set for compatibility with the multi-output code, but all reported test results use the exclusive four-way decoding above.

The split policy follows the usual train/development/test separation. Training data estimate templates or model weights. Development data choose block length, thresholds, and early stopping. Test data are used once for the final tables. No phone-level boundaries are supplied, so every method must find the useful initial-consonant evidence inside a whole-word recording.

1.1 Evaluation metrics

Let N be the number of test samples and $K = |C|$. Label-wise accuracy compares each class bit separately:

$$\text{LabelAcc} = \frac{1}{NK} \sum_{i=1}^{N} \sum_{c \in C} \mathbf{1}[\hat{y}_{i,c} = y_{i,c}].$$

For this one-hot task, label-wise accuracy is higher than ordinary accuracy because it also counts true negatives.

Exact-match accuracy requires the whole label vector to match:

$$\text{ExactMatch} = \frac{1}{N} \sum_{i=1}^{N} \mathbf{1}[\hat{y}_i = y_i].$$

Here it is the standard four-class accuracy, so the result tables treat it as the main metric.

For each label c, precision, recall, and F1 are

$$P_c = \frac{TP_c}{TP_c + FP_c}, \qquad R_c = \frac{TP_c}{TP_c + FN_c}, \qquad F1_c = \frac{2P_cR_c}{P_c + R_c}.$$

Macro-F1 averages the four class-level F1 scores:

$$\text{MacroF1} = \frac{1}{K} \sum_{c \in C} F1_c.$$

This metric shows whether one consonant class is being sacrificed.

Micro-F1 pools the TP, FP, and FN counts before computing F1:

$$\text{MicroF1} = \frac{2\sum_c TP_c}{2\sum_c TP_c + \sum_c FP_c + \sum_c FN_c}.$$

Samples-F1 computes F1 on each sample and then averages:

$$\text{SamplesF1} = \frac{1}{N} \sum_{i=1}^{N} \frac{2|Y_i \cap \hat{Y}_i|}{|Y_i| + |\hat{Y}_i|}.$$

Because every ground-truth and predicted label vector contains one positive class, micro-F1, samples-F1, and exact-match accuracy coincide numerically. They remain in the tables to match the shared evaluation interface.

The report is organized around the most interpretable method first. `strict_course_fft` is the course-constrained baseline: short-time FFT magnitude spectra and normalized correlation. `course_filterbank_corr` is reported as an extended template comparison because it adds log-mel features, onset localization, negative templates, and autocorrelation. The MLP and CNN appear after the signal-processing methods as baselines, not as the definition of the task.

2 Related Work

The task studied here is closer to local acoustic-event detection than to large-vocabulary speech recognition. The classifier only needs to decide which initial voiced consonant begins a short word. That makes short-time spectral analysis and template matching a useful starting point.

2.1 Short-time speech representations

Davis and Mermelstein showed that speech front ends gain stability by replacing raw waveforms with short-time parameterized spectra[1]. The usual pipeline frames the waveform over tens of milliseconds, applies a window, transforms each frame, and keeps the frequency-domain energy pattern. This representation preserves releases, frication, and vowel transitions while reducing sensitivity to raw waveform phase.

Rabiner's HMM tutorial connects frame-level features with temporal modeling[2]. The relevant point for this report is that speech evidence moves over time. Even without using an HMM, an initial-consonant detector should search a short region near the beginning of the word rather than average the whole recording into one global descriptor.

2.2 Why consonants need local evidence

The four target classes have different time-frequency behavior. /g/, /b/, and /d/ are voiced stops with short releases and fast transitions. /z/ is a voiced fricative with longer high-frequency energy. Whole-word averaging can bury these cues under the following vowel and speaker variation. Wilkinson and Russell's TIMIT phone-recognition work also points to the value of local formant and boundary-related evidence for phone discrimination[3].

For that reason, the main method starts with the tools closest to the course material: short-time FFT, local magnitude spectra, and normalized correlation. More complex baselines are useful only after the acoustic evidence has been made explicit.

3 Dataset Construction

3.1 Selection rule

The subset is generated by `src/prepare_mswc_initial_gbdz_subset.py`. A word is kept only when both conditions hold:

> The first letter is one of `g/b/d/z`, and the first non-vowel phone in CMUdict/ARPABET is the corresponding consonant `G/B/D/Z`.

This rule turns a loose spelling-based task into a pronunciation-based initial-consonant task. It excludes words where the target letter appears in the spelling but does not define the first consonant sound.

3.2 Corpus size

Table 1: Size of the pronunciation-filtered initial-consonant subset.

Label	Words	Train	Development	Test
/g/	15	600	75	75
/b/	15	600	75	75
/d/	15	600	75	75
/z/	15	600	75	75
Total	60	2400	300	300

The final corpus is balanced at the word and utterance levels. Each class contributes 15 words and 750 utterances, split as 600/75/75. The 300-sample test set therefore supports direct class-wise comparison without reweighting.

4 Methods

4.1 Course-constrained short-time FFT template method

`strict_course_fft` assumes that the initial consonant can be represented by a short local magnitude-spectrum prototype. Speaker identity, recording level, and the following vowel all introduce variation, so the method compares normalized spectral shape rather than raw energy.

For sample i, use frame length $L = 20$ ms, hop size $H = 10$ ms, and $N = 512$ FFT points. The spectrum of frame m is

$$X_{i,m}[k] = \sum_{n=0}^{L-1} x_i[n + mH]w[n]e^{-j2\pi kn/N}, \qquad k = 0, \ldots, N-1,$$

where $w[n]$ is a Hamming window. Only the one-sided magnitude spectrum is kept. For a candidate block starting at frame r and spanning M frames, the descriptor is

$$P_i^{(r)}[k] = \frac{1}{M} \sum_{m=0}^{M-1} \frac{|X_{i,r+m}[k]|}{\max_\kappa |X_{i,r+m}[\kappa]|}, \qquad k = 0, \ldots, N/2.$$

The block descriptor is then L_2-normalized.

Templates are estimated from the training data. For each class c, the method first chooses the maximum-energy M-frame block from each positive sample and calls it $P_i^{(\star)}$. The class template is the normalized average

$$T_c = \frac{\sum_{i:y_i=c} P_i^{(\star)}}{\left\| \sum_{i:y_i=c} P_i^{(\star)} \right\|_2}.$$

The development set selects the block length from $M \in \{3, 4, 5, 6\}$. The selected value is $M = 3$, after which the final templates are re-estimated from train+development samples. This keeps the main method within the course boundary: framing, FFT, averaging, and normalized correlation.

At test time, every candidate block is compared with every class template:

$$\rho_{i,c}^{(r)} = \frac{\langle P_i^{(r)}, T_c \rangle}{\|P_i^{(r)}\|_2 \|T_c\|_2}, \qquad s_{i,c} = \max_r \rho_{i,c}^{(r)}.$$

The predicted class is $\arg\max_c s_{i,c}$. Appendix A.1 gives the reproducible algorithm.

Figure 1 shows the template construction. The left side gives representative training blocks near the class mean, while the right side gives the normalized average template for each consonant.

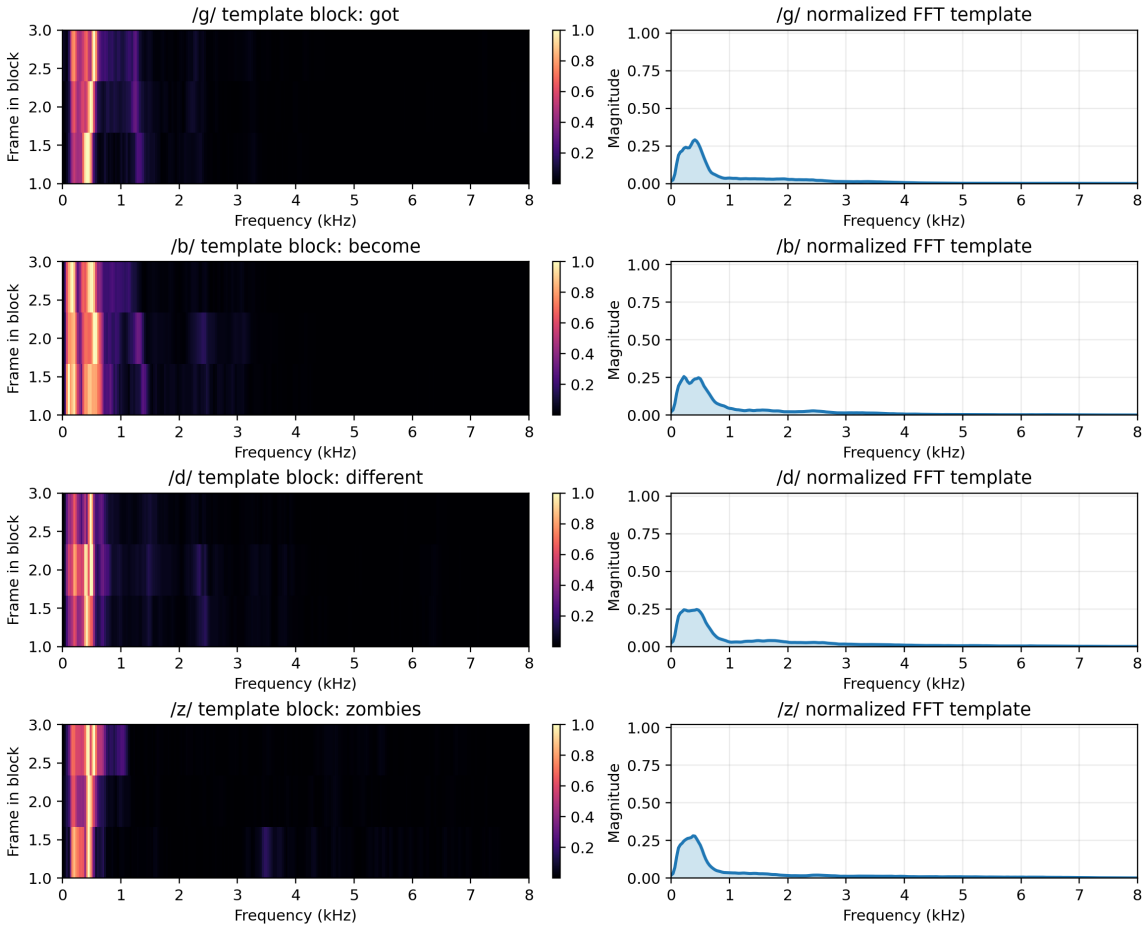

Figure 1: Template construction for `strict_course_fft`: maximum-energy blocks are extracted from train+development samples and averaged by class.

Figure 2 shows one correlation search. The method scans local candidate blocks, computes one score curve per class, and keeps the maximum score for each label.

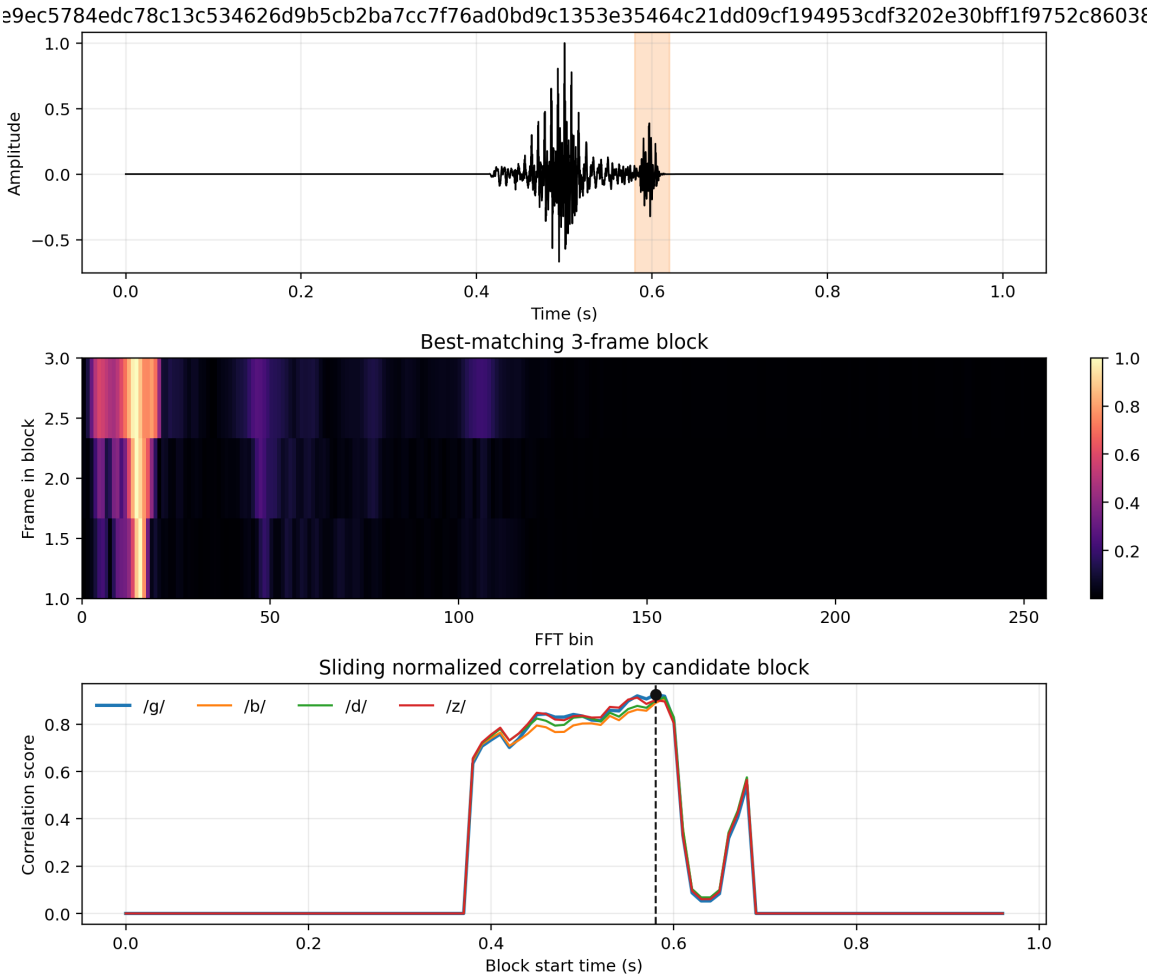

Figure 2: Correlation search in `strict_course_fft`. The classifier searches local blocks and assigns each class its best normalized-correlation score.

4.2 Extended template method with filterbanks and autocorrelation

`course_filterbank_corr` keeps the template-matching idea but expands the feature design. It uses log-mel filterbanks, estimates an onset window, builds both positive and negative templates for each label, and adds envelope autocorrelation. The method tests whether more precise localization and contrastive class information improve the same initial-consonant task.

With 25 ms frames and 10 ms hops, the method computes a 40-bin log-mel vector

$$L_i[m,q] = \log\left(\sum_k |X_{i,m}[k]|^2 B_q[k] + \epsilon\right), \qquad q = 1,\ldots,40.$$

Frame energy is

$$e_i[m] = \log\left(\frac{1}{40}\sum_q \exp L_i[m,q] + \epsilon\right).$$

A five-frame moving average smooths $e_i[m]$. The onset o_i is the first frame whose smoothed energy exceeds the initial median baseline by 0.22. The local window has 16 frames, starts two frames before the onset, and is evaluated under three offsets $\Delta \in \{-2,0,2\}$:

$$W_i^{(\Delta)} = L_i[o_i - 2 + \Delta : o_i - 2 + \Delta + 15, :].$$

Each candidate window is split into $S = 3$ segments. The feature vector concatenates raw segment means and segment means after subtracting the two-frame pre-onset baseline:

$$\phi_i^{(\Delta)} = \left[\mu_1(W), \mu_2(W), \mu_3(W), \mu_1(W - \bar{W}_{\text{pre}}), \mu_2(W - \bar{W}_{\text{pre}}), \mu_3(W - \bar{W}_{\text{pre}})\right].$$

Each μ_s is 40-dimensional, so $\phi_i^{(\Delta)} \in \mathbb{R}^{240}$. The same window also yields a 24-lag normalized autocorrelation vector $a_i^{(\Delta)}$ from the energy envelope.

For class c, the training set yields four templates: positive and negative spectral templates P_c^+ and P_c^-, plus positive and negative autocorrelation templates A_c^+ and A_c^-. The score for offset Δ is

$$\begin{aligned} g_{i,c}^{(\Delta)} &= 0.65\left[\cos(\phi_i^{(\Delta)}, P_c^+) - \cos(\phi_i^{(\Delta)}, P_c^-)\right] \\ &\quad + 0.35\left[\cos(a_i^{(\Delta)}, A_c^+) - \cos(a_i^{(\Delta)}, A_c^-)\right], \\ s_{i,c} &= \max_{\Delta \in \{-2,0,2\}} g_{i,c}^{(\Delta)}. \end{aligned}$$

The extra terms improve accuracy but make the method less course-constrained. For that reason, this report treats it as an extended comparison rather than the primary classroom method. Appendix A.2 gives the algorithm.

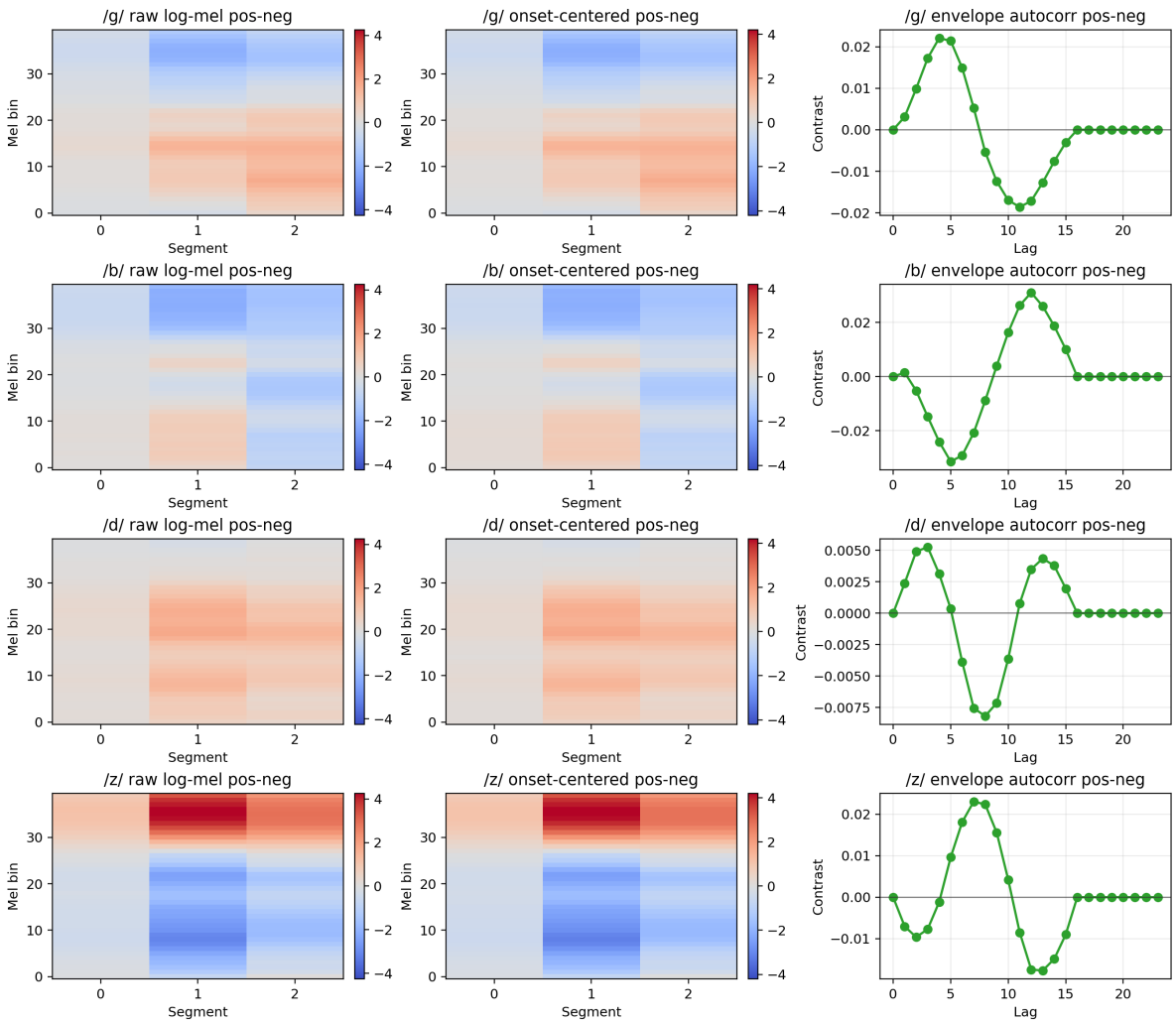

Figure 3: Positive-negative template differences in `course_filterbank_corr`. Warm regions are closer to positive samples; cool regions are closer to negative samples.

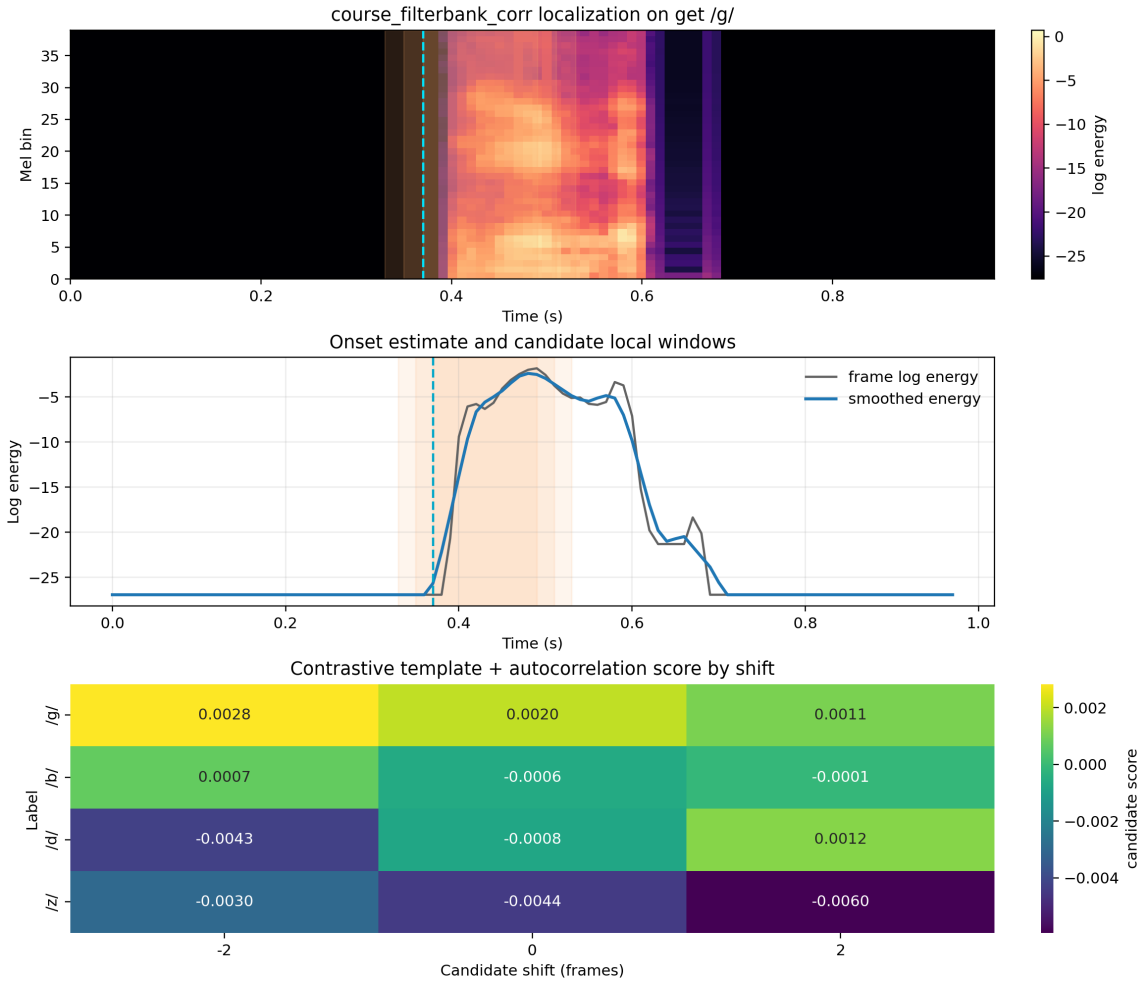

Figure 4: Onset localization and offset scoring in `course_filterbank_corr`.

4.3 MATLAB correspondence

The course-constrained method maps to `src/matlab_course_template_detector.m` and `src/matlab_course_spectrum_demo.m`. The core score is the normalized inner product

$$\texttt{score} = \frac{\texttt{mag}^\top \texttt{template}}{\|\texttt{mag}\|_2 \|\texttt{template}\|_2}.$$

`src/matlab_case_demo.m` visualizes the four case clips and their template scores. The MATLAB scripts show the course idea directly; the Python experiment adds development-set block selection and train+development template estimation for the formal test run.

5 Qualitative Case Analysis

5.1 Spectral templates for four representative words

Figure 5 shows the short-time FFT templates used in the MATLAB demonstration. The four representative words are *get*, *boy*, *did*, and *zero*. This figure is not a training summary; it is a visual check of the spectral shapes that the template method compares.

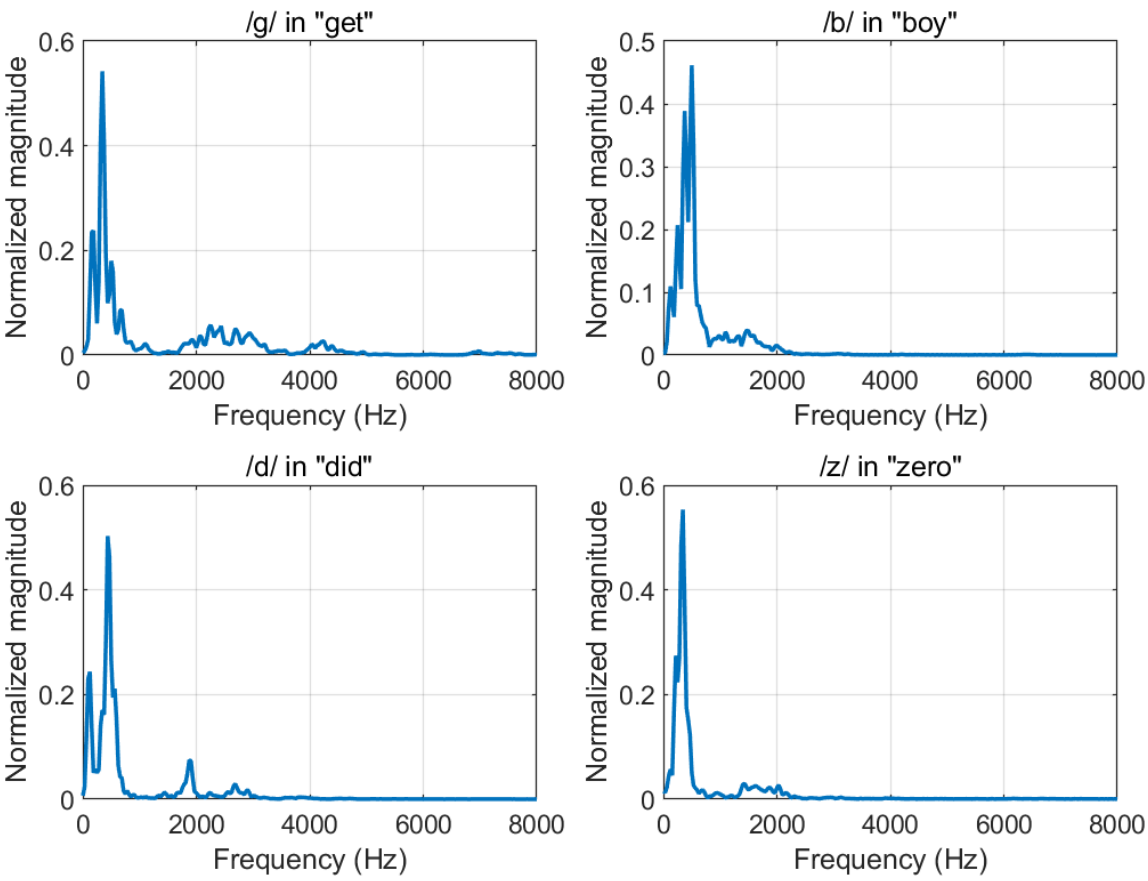

Figure 5: Short-time FFT magnitude templates for the four representative words.

The spectra match the expected acoustic pattern. /z/ has longer high-frequency frication. /b/ and /d/ depend on short release cues. /g/ is more sensitive to where the block is cut, which makes it easier for alignment error to blur the evidence.

5.2 Correlation score matrix

Figure 6 reports the correlation scores between the four case words and the four templates. The matrix turns template similarity into a numerical object, making it clear when the correct template wins and when the margins are small.

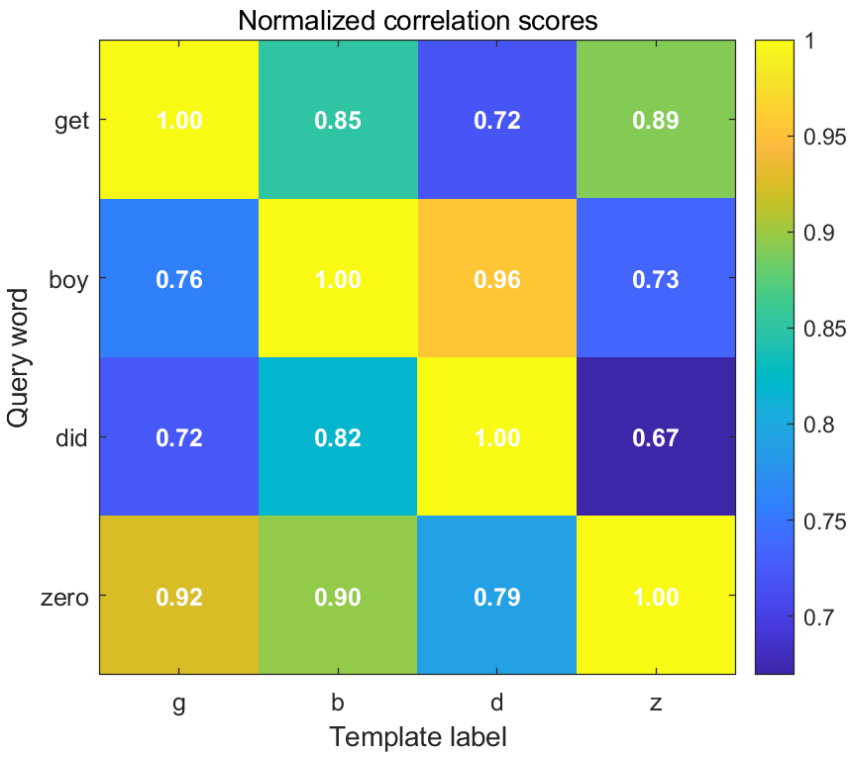

Figure 6: Correlation score matrix for the four representative words.

Small gaps between the top scores usually indicate unstable onset alignment or a weak release cue. This observation motivates the extended method's onset-centered search, but it also explains why the strict FFT baseline remains useful for interpretation.

6 Experimental Results

6.1 Course-constrained baseline

The strict FFT template method is evaluated first because it is the main signal-processing baseline. In a one-hot four-class task, exact-match accuracy is the direct metric; label-wise accuracy is included only for consistency with the shared evaluation code.

The strict method uses two course-compatible refinements. It no longer builds templates from only the four case clips. Instead, it averages maximum-energy blocks from same-class training samples and then re-estimates the templates on train+development after choosing the block length. The development set selects a three-frame block.

Table 2: Main metrics for the course-constrained FFT template method.

Method	Label acc.	Exact match	Macro-F1	Micro-F1	Samples-F1
strict_course_fft	0.6717	0.3433	0.3368	0.3433	0.3433

Table 3: Per-label F1 for the course-constrained FFT template method.

Method	/g/	/b/	/d/	/z/
strict_course_fft	0.250	0.377	0.345	0.375

Figure 7 shows that positive and negative score distributions are separated but still overlap. The strict method captures meaningful spectral evidence, but the evidence is not strong enough for stable four-class recognition.

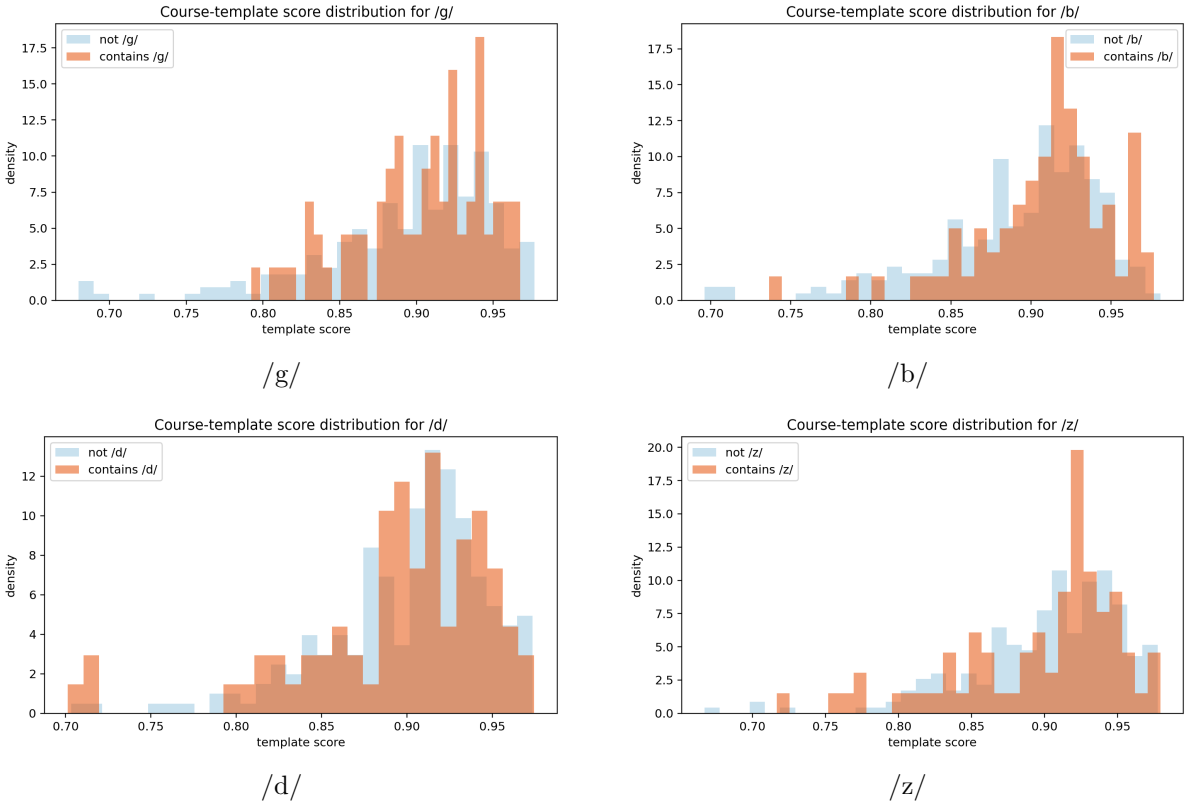

Figure 7: Score distributions from `strict_course_fft` for the four labels.

6.2 Extended template comparison

`course_filterbank_corr` adds onset localization, log-mel positive-negative template differences, and envelope autocorrelation. These changes move beyond the strict course boundary, so the method is reported as an extended comparison.

Table 4: Overall comparison between the course-constrained and extended template methods.

Method	Label acc.	Exact match	Macro-F1	Micro-F1	Samples-F1
strict_course_fft	0.6717	0.3433	0.3368	0.3433	0.3433
course_filterbank_corr	0.7750	0.5500	0.5462	0.5500	0.5500

Table 5: Per-label F1 for the course-constrained and extended template methods.

Method	/g/	/b/	/d/	/z/
strict_course_fft	0.250	0.377	0.345	0.375
course_filterbank_corr	0.438	0.528	0.472	0.747

The extended method improves exact-match accuracy from 0.343 to 0.550. The largest gain appears for /z/, whose sustained frication is well matched by log-mel local templates

and autocorrelation. /g/ remains the weakest class, consistent with its short burst and higher alignment sensitivity.

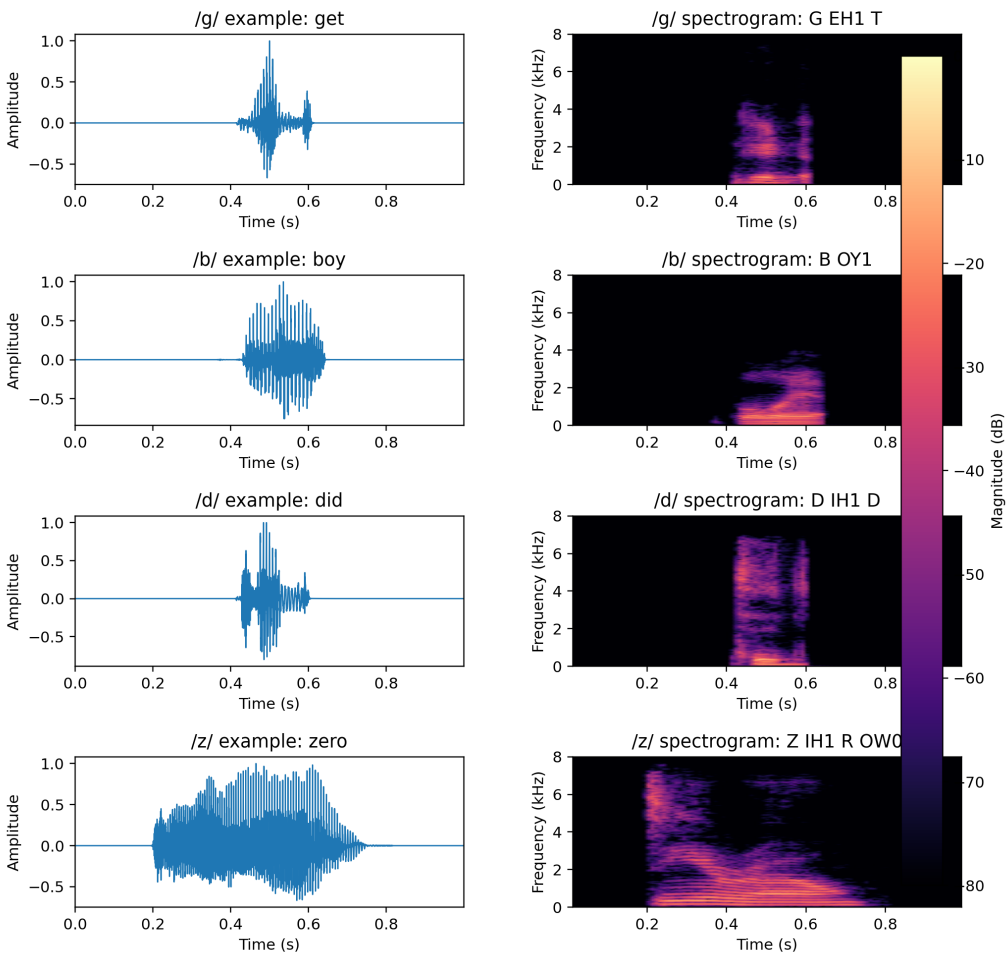

Figure 8: Representative examples used by the extended template pipeline.

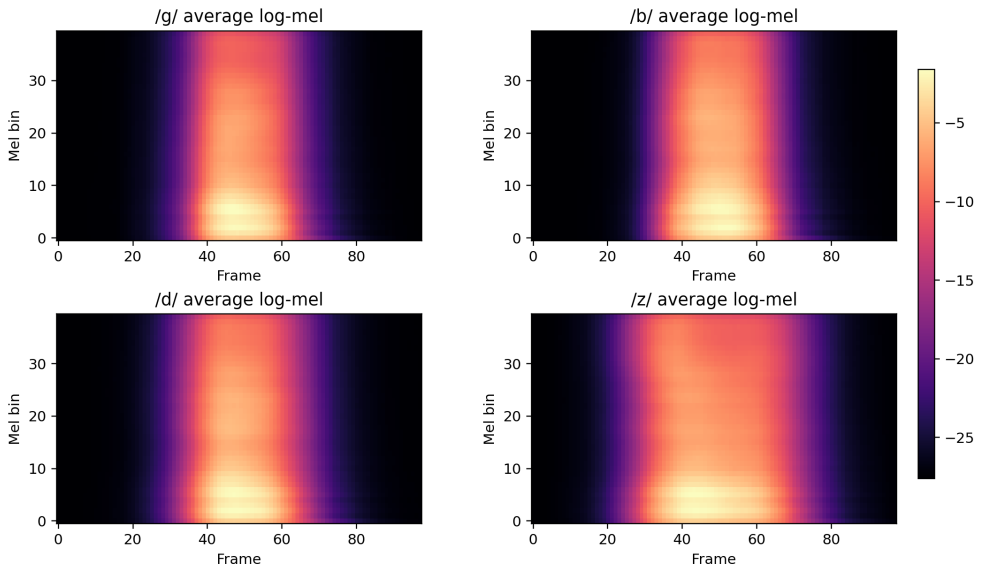

Figure 9: Average log-mel templates for the four classes. This view shows class-level energy patterns; it is separate from the positive-negative contrast templates used in scoring.

Figure 10 gives the test score distributions for the extended method. /z/ shows the clearest positive-negative separation. /g/, /b/, and /d/ still overlap, which explains why onset localization helps but does not remove stop-consonant confusion.

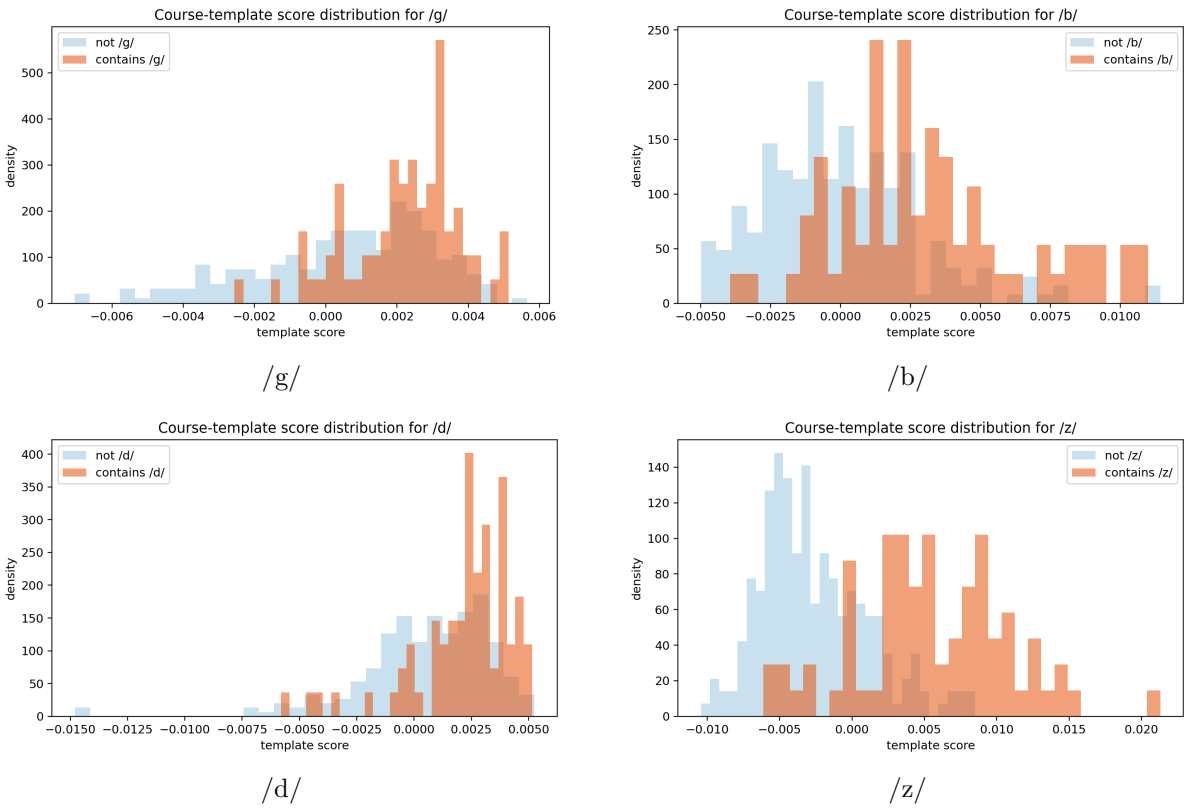

Figure 10: Score distributions from `course_filterbank_corr` for the four labels.

7 Neural and Statistical Baselines

The baselines in this section compare the template methods with two learned classifiers: a feature-based MLP and a small CNN. Both models train on the training split, use the development split for thresholding or early stopping, and report final metrics on the test split. Although the task is one-hot four-way classification, both models keep the same four-output interface as the template methods and use arg max for the final label.

7.1 MLP on MFCC and band-energy statistics

The MLP does not take a full spectrogram as input. It uses a fixed vector of summary features. For each recording, the pipeline computes log-mel features, extracts 13 MFCC coefficients and their first-order differences, and concatenates

$$z_i = [\text{mean}(\text{MFCC}), \text{std}(\text{MFCC}), \text{mean}(\Delta\text{MFCC}), \text{std}(\Delta\text{MFCC}), \text{bandEnergy}] .$$

The first four groups contribute $13 \times 4 = 52$ dimensions. The band-energy block contributes 10 more, so $z_i \in \mathbb{R}^{62}$. The features are standardized with the training-set mean and variance.

The network has two hidden layers:

$$h_1 = \text{ReLU}(W_1 z_i + b_1), \qquad h_2 = \text{ReLU}(W_2 h_1 + b_2), \qquad p_i = \sigma(W_3 h_2 + b_3),$$

with widths 128 and 64. The loss is binary cross entropy over the four output labels plus L_2 regularization:

$$\mathcal{L}_{\text{MLP}} = -\sum_{i,c} [y_{i,c} \log p_{i,c} + (1 - y_{i,c}) \log(1 - p_{i,c})] + \lambda \|\theta\|_2^2.$$

The MLP tests whether whole-word MFCC statistics already contain enough class information. It does not localize the initial consonant, so vowel and speaker differences remain mixed with the target cue. Appendix A.3 gives the algorithm.

7.2 CNN on whole-word log-mel spectrograms

The CNN receives a whole-word log-mel spectrogram $L_i \in \mathbb{R}^{T \times 40}$. The input is standardized by the training-set global mean and standard deviation and then passed as a single-channel image. The architecture is

$$\text{Conv}(1, 24) \to \text{BN} \to \text{ReLU} \to \text{MaxPool},$$

$$\text{Conv}(24, 48) \to \text{BN} \to \text{ReLU} \to \text{MaxPool},$$

$$\text{Conv}(48, 96) \to \text{BN} \to \text{ReLU} \to \text{GlobalAvgPool} \to \text{FC}(96) \to \text{Dropout}(0.25) \to \text{FC}(4).$$

Let the logits be $u_{i,c}$ and probabilities be $p_{i,c} = \sigma(u_{i,c})$. The loss is weighted binary cross entropy:

$$\mathcal{L}_{\text{CNN}} = -\sum_{i,c} \alpha_c y_{i,c} \log p_{i,c} - (1 - y_{i,c}) \log(1 - p_{i,c}),$$

where α_c is computed from the positive/negative ratio in the training split and clipped to $[1, 10]$. Training uses AdamW with learning rate 10^{-3}, weight decay 10^{-4}, at most 10 epochs, and development macro-F1 for early stopping.

The CNN lets the model learn local patterns from the whole spectrogram. This assumption needs more data than the 3000-utterance, 60-word corpus provides, and the model is not forced to attend to the word onset.

Table 6: Overall metrics for the learned baselines.

Method	Label acc.	Exact match	Macro-F1	Micro-F1	Samples-F1
MLP	0.7633	0.5267	0.5258	0.5267	0.5267
CNN	0.6600	0.3200	0.2219	0.3200	0.3200

The MLP is a useful statistical baseline, but it remains below the extended template method. The CNN trains successfully but performs worse than the strict FFT template on exact-match accuracy. In this dataset, a small end-to-end spectrogram model does not recover the onset cue more reliably than a hand-designed local spectral comparison.

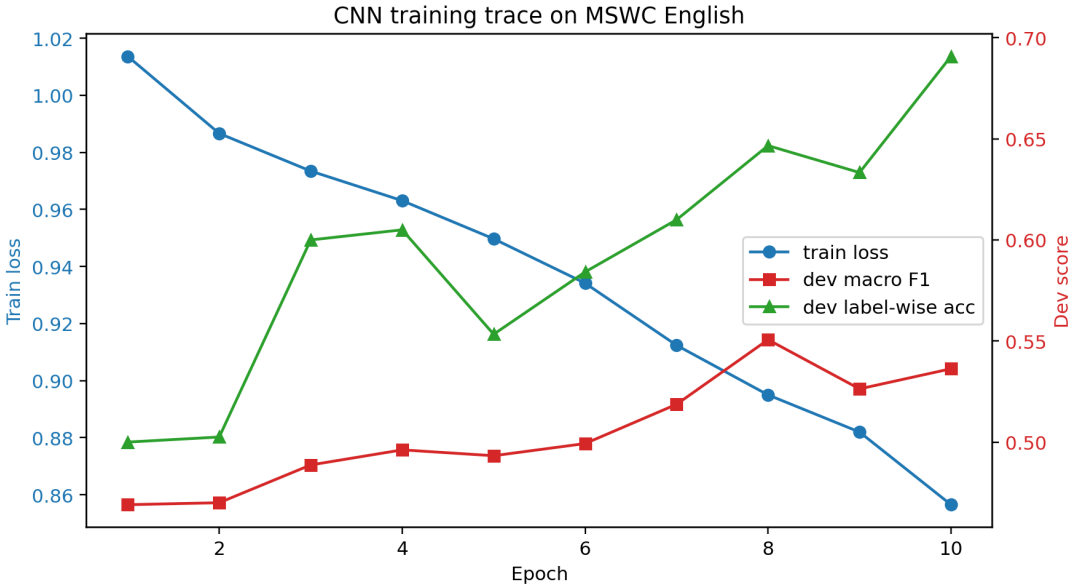

Figure 11: CNN training history.

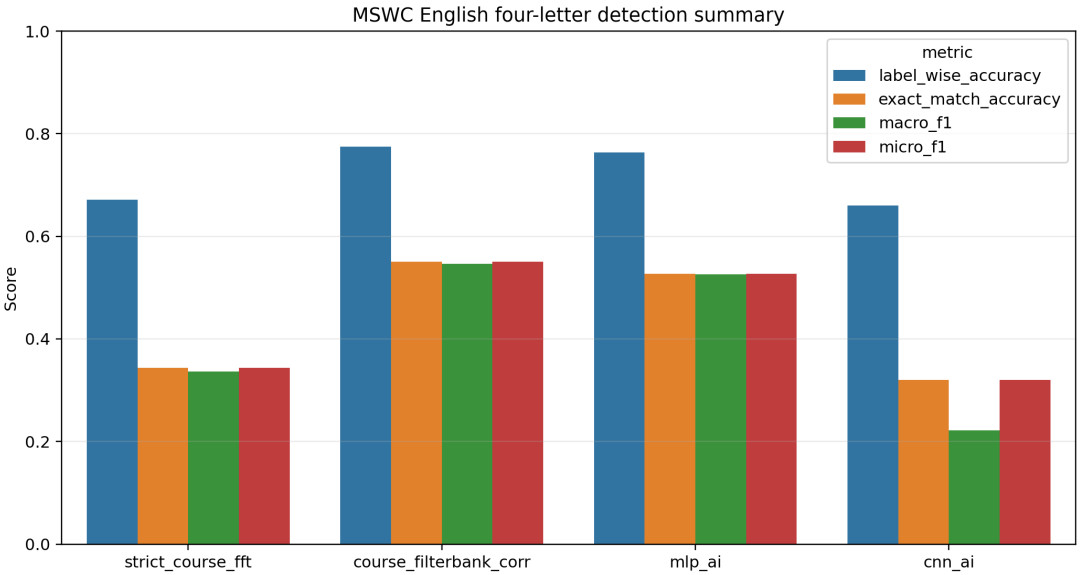

Figure 12: Overall metric summary for all methods.

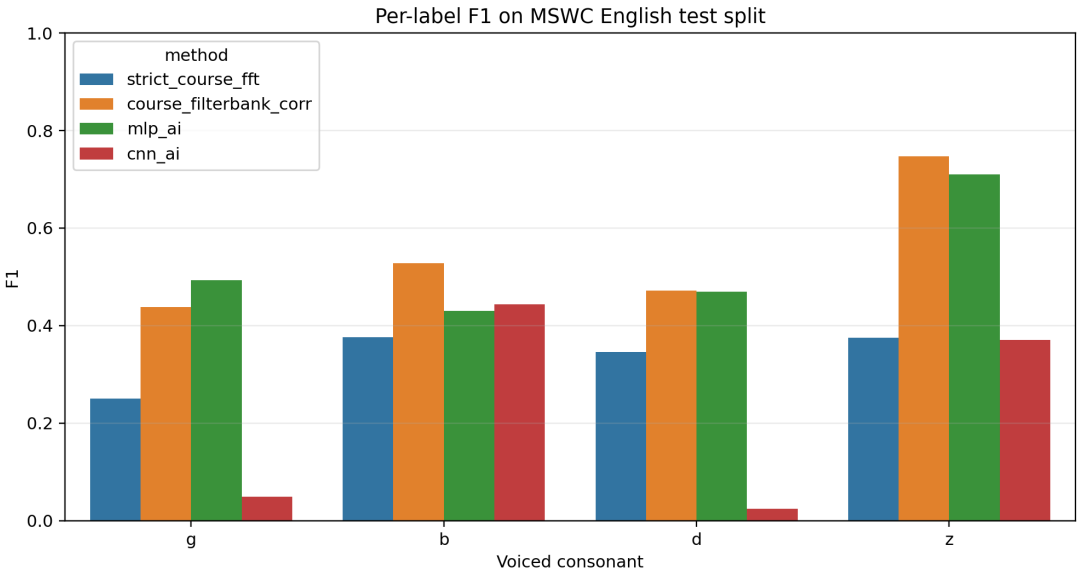

Figure 13: Per-label F1 comparison across all methods.

8 Discussion and Limitations

The reconstructed task separates three roles. The strict FFT template method demonstrates what the course-level tools can explain. The extended template method measures how much accuracy improves after adding onset localization, contrastive templates, and autocorrelation. The MLP and CNN provide learned baselines for comparison.

The main limitation is that the input remains a whole-word recording rather than a manually segmented phone. The strict method therefore relies on sliding-window search, and short stop bursts can be missed when the chosen block is slightly misaligned. This affects /g/ and /d/ most strongly. /z/ benefits from longer frication, so it gives the cleanest score separation under both template methods.

These results argue for a layered evaluation. A higher score alone does not explain which acoustic cue the method used. The strict template method gives the clearest view of the local spectral evidence; the extended method improves recognition by adding localization and contrastive statistics; the learned baselines show that more flexible models are not automatically better on a small, onset-sensitive corpus.

9 Conclusion

The study rebuilds VE216 Topic 2 as a pronunciation-filtered initial-consonant detection task over /g/, /b/, /d/, and /z/. The resulting MSWC English subset contains 60 words and 3000 utterances with balanced train, development, and test splits.

The course-constrained method, `strict_course_fft`, uses only short-time FFT magnitude spectra and normalized correlation. With a three-frame block selected on the development set and templates re-estimated from train+development data, it reaches 0.343 exact-match accuracy. The extended template method, `course_filterbank_corr`, improves exact-match accuracy to 0.550 by adding log-mel onset localization, positive-negative templates, and envelope autocorrelation. The MLP reaches 0.527, while the CNN reaches 0.320.

The class-level pattern is consistent across the analysis: /z/ is easiest because its frication is longer and more stable, and /g/ is hardest because its useful burst evidence is brief and alignment-sensitive. The final comparison supports the main design choice of the report: start with an interpretable spectral template method, then use stronger baselines to measure what the extra modeling buys.

A Algorithmic Details

This appendix summarizes the four implementations used in the report. All methods use 16 kHz, one-second mono WAV files, the label set $C = \{g, b, d, z\}$, and the same train/development/test roles.

A.1 `strict_course_fft`

Assumption. Each initial voiced consonant can be represented by a short-time FFT magnitude-spectrum prototype. A maximum-energy block approximates the local evidence for the initial consonant, and normalized correlation reduces level differences.

Procedure.

1. Frame each training waveform with 20 ms windows, 10 ms hops, and a 512-point FFT.
2. Compute the normalized magnitude-spectrum descriptor $P_i^{(r)}$ for each candidate M-frame block.
3. For each positive training sample, select the maximum-energy block $P_i^{(\star)}$.
4. For each class c, average same-class $P_i^{(\star)}$ descriptors and normalize the result to obtain T_c.
5. Select $M \in \{3, 4, 5, 6\}$ on the development set by exact-match accuracy.
6. Re-estimate templates on train+development using the selected block length.
7. At test time, compute $s_{i,c} = \max_r \cos(P_i^{(r)}, T_c)$ over all candidate blocks.
8. Predict $\hat{c}_i = \arg\max_c s_{i,c}$.

A.2 `course_filterbank_corr`

Assumption. The onset neighborhood is more reliable than whole-word averaging. Positive-negative template differences reduce shared spectral energy, and envelope autocorrelation helps separate sustained frication from short stop releases.

Procedure.

1. Frame the audio with 25 ms windows and 10 ms hops, then compute a 40-bin log-mel spectrogram.
2. Estimate onset o_i from the smoothed frame-energy curve.
3. Extract 16-frame candidate windows using offsets $\Delta \in \{-2, 0, 2\}$.
4. Split each window into three segments and concatenate raw segment means with baseline-subtracted segment means to form $\phi_i^{(\Delta)}$.
5. Compute a 24-lag normalized autocorrelation vector $a_i^{(\Delta)}$ from the same window's energy envelope.
6. For each class, average positive and negative training features to obtain P_c^+, P_c^-, A_c^+, and A_c^-.
7. Score each offset with $0.65[\cos(\phi, P_c^+) - \cos(\phi, P_c^-)] + 0.35[\cos(a, A_c^+) - \cos(a, A_c^-)]$.
8. Use the maximum score over the three offsets as $s_{i,c}$, then predict $\arg\max_c s_{i,c}$.

A.3 `mlp_ai`

Assumption. MFCC means, standard deviations, delta-MFCC statistics, and coarse band energies summarize enough whole-word acoustic information for a two-layer nonlinear classifier.

Structure.

$$62 \to 128 \to 64 \to 4.$$

Hidden layers use ReLU activations, and the output layer uses sigmoid probabilities. Training uses Adam, early stopping, and L_2 regularization. The development set selects thresholds, while test-time four-way classification uses $\arg\max$.

Procedure.

1. Extract MFCC statistics, delta-MFCC statistics, and 10-dimensional band energy from each recording.
2. Standardize features using the training-set mean and variance.
3. Train the two-layer MLP with binary cross entropy over four labels.
4. Estimate per-label thresholds on the development set while preserving exclusive decoding.
5. On the test set, output four probabilities and choose the largest one.

A.4 `cnn_ai`

Assumption. A log-mel spectrogram can be treated as a time-frequency image, and convolutional layers can learn useful local acoustic patterns without a hand-specified onset window. The assumption requires enough data for the model to ignore irrelevant positions.

Structure.

$$\begin{aligned}
&\mathrm{Conv}_{24} \to \mathrm{BN} \to \mathrm{ReLU} \to \mathrm{Pool} \\
&\to \mathrm{Conv}_{48} \to \mathrm{BN} \to \mathrm{ReLU} \to \mathrm{Pool} \\
&\to \mathrm{Conv}_{96} \to \mathrm{BN} \to \mathrm{ReLU} \to \mathrm{GlobalAvgPool} \\
&\to \mathrm{FC}_{96} \to \mathrm{Dropout} \to \mathrm{FC}_{4}.
\end{aligned}$$

Procedure.

1. Extract whole-word log-mel spectrograms and standardize them with the training-set global mean and variance.
2. Feed the spectrograms as single-channel inputs to the three-stage convolutional network.
3. Train with positive-weighted `BCEWithLogitsLoss` and AdamW.
4. After each epoch, compute development macro-F1 and keep the best checkpoint.
5. On the test set, apply sigmoid to the logits and choose the largest probability class.

References

[1] S. Davis and P. Mermelstein, "Comparison of Parametric Representations for Monosyllabic Word Recognition in Continuously Spoken Sentences," *IEEE Transactions on Acoustics, Speech, and Signal Processing*, 1980. https://doi.org/10.1109/TASSP.1980.1163420.

[2] L. R. Rabiner, "A Tutorial on Hidden Markov Models and Selected Applications in Speech Recognition," *Proceedings of the IEEE*, 1989. https://doi.org/10.1109/5.18626.

[3] N. J. Wilkinson and M. J. Russell, "Improved Phone Recognition on TIMIT Using Formant Frequency Data and Confidence Measures," in *Proc. ICSLP 2002*. https://www.isca-archive.org/icslp_2002/wilkinson02_icslp.html.

[4] G. Chen, C. Parada, and G. Heigold, "Small-Footprint Keyword Spotting using Deep Neural Networks," in *Proc. ICASSP 2014*. https://research.google/pubs/small-footprint-keyword-spotting-using-deep-neural-networks/.

[5] T. N. Sainath and C. Parada, "Convolutional Neural Networks for Small-Footprint Keyword Spotting," in *Proc. Interspeech 2015*. https://research.google/pubs/convolutional-neural-networks-for-small-footprint-keyword-spotting/.

[6] P. Warden, "Speech Commands: A Dataset for Limited-Vocabulary Speech Recognition," arXiv:1804.03209, 2018. https://doi.org/10.48550/arXiv.1804.03209.

[7] M. Berg *et al.*, "Keyword Transformer: A Self-Attention Model for Keyword Spotting," arXiv:2104.00769, 2021.

[8] A. Baevski *et al.*, "wav2vec 2.0: A Framework for Self-Supervised Learning of Speech Representations," arXiv:2006.11477, 2020. https://doi.org/10.48550/arXiv.2006.11477.

[9] W.-N. Hsu *et al.*, "HuBERT: Self-Supervised Speech Representation Learning by Masked Prediction of Hidden Units," arXiv:2106.07447, 2021.

[10] S. Chen *et al.*, "WavLM: Large-Scale Self-Supervised Pre-Training for Full Stack Speech Processing," *IEEE Journal of Selected Topics in Signal Processing*, 2022. https://arxiv.org/abs/2110.13900.

[11] C. Gao *et al.*, "Self-Supervised Speech Representation Learning for Keyword-Spotting with Light-Weight Transformers," arXiv:2303.04255, 2023. https://arxiv.org/abs/2303.04255.

[12] D. Kim and J. Hwang, "Fully End-to-end Streaming Open-vocabulary Keyword Spotting with W-CTC Forced Alignment," in *Proc. Interspeech 2025*. https://www.isca-archive.org/interspeech_2025/kim25d_interspeech.html.